

Functions (Focused on return type)

Author: Subhasis Das

Date: 17/02/2024

Function Structure:

```
access-specifier static/non-static return_type function_name(parameters/arguments){  
//function body  
}
```

**** access specifier** is an optional definition: there are three access specifiers: **private**, **protected** and **public**.

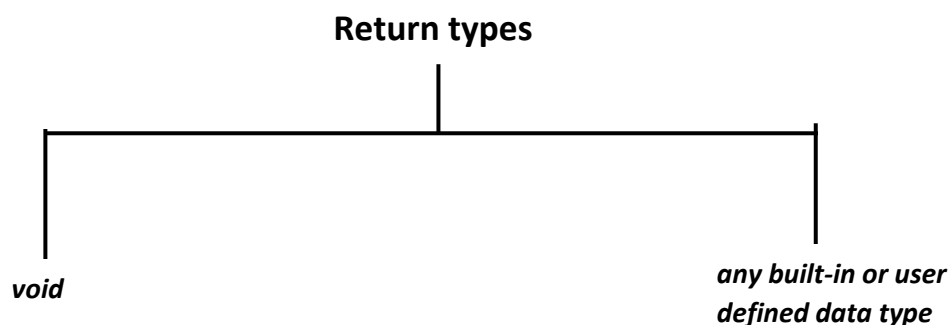
******Defining a function as **public** makes it accessible from outside classes.

******Using **private** as access specifier makes it accessible only within its own class.

******By using **protected** as access specifier makes it accessible by its own class and by subclasses of its class.

******* a default access specifier called **default** is automatically used when the access specifier isn't defined by programmer

****** Defining a function as **static** or not decides its memory location. we have discussed this in the previous document- "Memory Occupation in JAVA"



The **void** keyword is used when there is nothing to return. With **void** used at the function header, the function will simply execute and will not return anything.

Now, When I say, "**any built in or user defined types**", It means, any built-in type like

Primitive types: **byte, char, short, int, long, float, double** or any non-primitive type:

String, array, class, enum etc...

Example 1 (functions with the keyword **void**)

```
class Apple{
int n;
void input(){

    System.out.println("Enter a number");
    Scanner sc=new Scanner(System.in);
    n=sc.nextInt();
}

void factorial(){
    if(n==0){
        System.out.println("Factorial=1");

    }else{
        int p=1;
        for(int i=1; i<=n; i++){
            p*=i;
        }
        System.out.println("Factorial="+p);
    }

}

public static void main(String args[]){
    Apple obj=new Apple();
    obj.input();
    obj.factorial();
}

}
```

****This program will take a number from user and simply print its factorial.**

Now if we want to make a function return something, we must know what type of data it will return, the very same data type will be used as return type.

For example, if we want a function to add two numbers of integer type, we can use **int** for this case. Assuming we already have two data members **a** and **b** in our class.

```
int sum(){
    int s=a+b;
    return s;
}
```

**** Remember:** return type of a function must be same as the data type of returning value or variable. Also the return type of a function can be larger/wider than the data type of returning value of variable but the opposite isn't allowed to avoid data loss. Here is an example:

```
double sum(){
    int s=a+b;
    return s;
}
```

The above code is possible.

```
int sum(){
    double s=a+b;
    return s;
}
```

This code will throw error as double can't be converted to int automatically. However, we can use type casting to make it possible, although a value loss can occur.

With casting:

```
int sum(){
    double s=a+b;
    return (int)s;
}
```

Example 2: (A function that can return a variable)

```
class Apple{
    int n;
    void input(){

        System.out.println("Enter a number");
        Scanner sc=new Scanner(System.in);
        n=sc.nextInt();
    }
}
```

```

int factorial(){
    if(n==0){
        return 1;

    }else{
        int p=1;
        for(int i=1; i<=n; i++){
            p*=i;
        }
        return p;
    }
}

public static void main(String args[]){
    Apple obj=new Apple();
    obj.input();
    int k=obj.factorial();
}
}

```

** Here the function call ***obj.factorial()*** will return ***n!*** (factorial of ***n***) where ***n*** is a value given by the user. And the returned value will be stored in ***k***.

*if a function ***A()*** is called within another function definition ***B()*** , calling ***B()*** will also call ***A()***

** Calling a function within itself is called **recursion** which is similar to iteration (loop)